



Rapport de Projet de Fin d'Études

PFE n°1 — SLAM Multi-Capteurs

MC-SLAM

[Matis DUVAL]

Année universitaire 2025–2026

Résumé

Ce rapport présente le travail réalisé dans le cadre du [Projet de Fin d'Études \(PFE\)](#) intitulé *SLAM Multi-Capteurs (MC-SLAM)*. L'objectif principal est la prise en main de la plateforme TurtleBot 4 sous [Robot Operating System \(ROS\) 2](#), suivi du développement d'une solution de [Simultaneous Localization and Mapping \(SLAM\)](#) exploitant une caméra [RGB](#) et un [LiDAR](#).

Table des matières

1	Introduction	5
1.1	Contexte	5
1.2	Problématique	5
1.3	Sujet et objectifs	6
2	État de l’art	7
2.1	SLAM visuel (Visual SLAM)	7
2.2	SLAM LiDAR	7
2.3	SLAM multi-capteurs	8
2.4	SLAM thermique (LWIR)	8
2.5	3D Gaussian Splatting	8
3	User Stories	9
4	Spécification des besoins	10
4.1	Besoins fonctionnels	10
4.2	Besoins non fonctionnels	10
4.3	Cas de tests	11
5	Choix logiciels	12
5.1	Système d’exploitation et middleware	12
5.2	Algorithmes de SLAM	12
5.3	Calibration	12
5.4	Évaluation	13
5.5	Développement et versioning	13
6	Architecture	14
6.1	Architecture matérielle	14
6.2	Architecture logicielle ROS 2	14
6.3	Diagramme de cas d’utilisation (UML)	15

7	Réalisation	16
7.1	Mise en place de l'environnement	16
7.2	Screenshots	16
7.3	Algorithmes et méthodes	17
7.3.1	Pipeline SLAM RTAB-Map	17
7.3.2	Calibration extrinsèque caméra/LiDAR	17
8	Résultats et tests	18
8.1	Résultats expérimentaux	18
9	Conclusion	19
9.1	Bilan	19
9.2	Perspectives	19
	Références bibliographiques	20

Table des figures

6.1	Graphe des nœuds et topics ROS 2 du pipeline MC-SLAM	14
6.2	Diagramme de cas d'utilisation UML	15
7.1	Visualisation des cartes générées par les deux pipelines SLAM	16

Liste des tableaux

3.1	User stories principales	9
4.1	Plan de tests fonctionnels	11
8.1	Comparaison des performances SLAM sur la séquence de test	18

Chapitre 1

Introduction

1.1 Contexte

La robotique mobile autonome connaît un essor considérable, portée par les avancées en intelligence artificielle, en vision par ordinateur et en capteurs embarqués. Dans ce contexte, la capacité d'un robot à se localiser tout en construisant une carte de son environnement, le [Simultaneous Localization and Mapping \(SLAM\)](#) est devenu une brique fondamentale de nombreuses applications: navigation en intérieur, véhicules autonomes, inspection industrielle ou encore robotique de service.

Ce Projet vise à exploiter la plateforme robotique **TurtleBot 4** pour développer et comparer différentes approches de [SLAM](#) multi-capteurs.

1.2 Problématique

Malgré la maturité des algorithmes de [SLAM](#) monoculaires ou monocapteurs, plusieurs verrous subsistent:

- **Robustesse aux conditions d'éclairage:** les approches [RGB](#) sont sensibles aux variations lumineuses; les capteurs [LWIR](#) offrent une alternative thermique mais restent peu explorés dans ce contexte.
- **Fusion multi-capteurs :** la combinaison [LiDAR](#) + caméra peut améliorer la précision, mais nécessite une calibration rigoureuse.
- **Représentation dense de l'environnement:** les cartes éparses issues des approches classiques sont limitées pour la navigation fine et la reconstruction 3D photo-réaliste.

1.3 Sujet et objectifs

Les objectifs du projet sont les suivants:

1. **Prise en main** de la plateforme TurtleBot 4 (architecture matérielle, logicielle sous [ROS 2](#), calibration des capteurs).
2. **Développement d'un pipeline MC-SLAM** exploitant la caméra [RGB](#) et/ou le [LiDAR](#) intégré.
3. **Extensions (bonus)** : implémentation d'un SLAM [LWIR](#) et intégration du *3D Gaussian Splatting* pour une représentation dense et photo-réaliste.

Chapitre 2

État de l'art

2.1 SLAM visuel (Visual SLAM)

Le SLAM visuel utilise des caméras comme capteur principal. Parmi les approches les plus répandues:

ORB-SLAM3 Système complet supportant caméras monoculaire, stéréo et RGB-D, avec gestion de la réinitialisation et des cartes multi-sessions.[\[1\]](#).

DSO (Direct Sparse Odometry) Approche directe minimisant l'erreur photométrique, robuste aux scènes peu texturées.

VINS-Mono / VINS-Fusion Intégration vision + [IMU](#) pour une odométrie visuo-inertielle précise.

2.2 SLAM LiDAR

Les approches LiDAR offrent une précision métrique supérieure en extérieur et dans les larges espaces:

LOAM / LeGO-LOAM Extraction de *features* géométriques (arêtes, plans) pour l'odométrie et la cartographie en temps réel.

LIO-SAM Fusion LiDAR + [IMU](#) avec optimisation par graphe de facteurs.

KISS-ICP Pipeline minimaliste et robuste basé sur l'ICP adaptatif, particulièrement adapté aux plateformes embarquées.

2.3 SLAM multi-capteurs

La fusion de plusieurs modalités permet de combiner leurs forces respectives. Les travaux récents s'appuient sur des graphes de facteurs (GTSAM, g2o) pour intégrer des contraintes hétérogènes (LiDAR, caméra, IMU, GPS).

2.4 SLAM thermique (LWIR)

Le SLAM basé sur caméra LWIR est un domaine émergent. Les images thermiques sont invariantes aux conditions d'éclairage visible, ce qui les rend attractives pour les environnements obscurs ou enfumés. Cependant, leur faible résolution et le manque de texture fine posent des défis pour l'extraction de *keypoints*. Des avancées récentes comme le Thermal-SLAM qui réduit l'impact du bruit thermique et améliore la détection de *features*[2].

2.5 3D Gaussian Splatting

Introduit par Kerbl *et al.* (2023), le 3D Gaussian Splatting (3DGS) représente une scène 3D par un ensemble de gaussiennes 3D anisotropes, permettant un rendu en temps réel de haute qualité. Son intégration dans un pipeline SLAM (*SplaTAM*, *MonoGS*) ouvre la voie à des cartes denses et photo-réalistes.

Chapitre 3

User Stories

Les *user stories* suivantes ont été définies par mes soins pour permettre une meilleure compréhension de l'objectif principal et afin de créer un Kanban.

Table 3.1: User stories principales

ID	Rôle	Objectif	Priorité
US-01	Opérateur	Démarrer le TurtleBot 4 et lancer un nœud ROS 2 de base	Haute
US-02	Opérateur	Calibrer la caméra RGB et le LiDAR (intrinsèque + extrinsèque)	Haute
US-03	Ingénieur	Lancer un pipeline SLAM RGB et visualiser la carte en temps réel dans RViz2	Haute
US-04	Ingénieur	Lancer un pipeline SLAM LiDAR et comparer la précision avec US-03	Haute
US-05	Ingénieur	Fusionner les données RGB + LiDAR pour améliorer la robustesse du SLAM	Moyenne
US-06	Chercheur	Remplacer la caméra RGB par une caméra LWIR et évaluer la précision du SLAM dans l'obscurité	Basse (bonus)
US-07	Chercheur	Intégrer le 3D Gaussian Splatting dans le pipeline SLAM pour générer une carte dense photo-réaliste	Basse (bonus)

Chapitre 4

Spécification des besoins

4.1 Besoins fonctionnels

- BF-1** Le système doit permettre le pilotage du TurtleBot 4 via une téléopération [ROS 2](#) (clavier ou joystick).
- BF-2** Le système doit acquérir et synchroniser les flux de données de la caméra RGB-D (OAK-D Pro) et du LiDAR (RPLiDAR A1M8).
- BF-3** Le système doit réaliser la calibration intrinsèque de la caméra (modèle pinhole + distorsion) et la calibration extrinsèque caméra/LiDAR.
- BF-4** Le système doit exécuter un algorithme de SLAM RGB en temps réel et publier la carte et la pose estimée sur les topics ROS 2 adéquats.
- BF-5** Le système doit exécuter un algorithme de SLAM LiDAR en temps réel.
- BF-6** Le système doit permettre l'évaluation quantitative de la précision (ATE, RPE) via les outils `evo`.
- BF-7 (Bonus)** Le système doit supporter l'acquisition de données LWIR et exécuter un pipeline SLAM thermique.
- BF-8 (Bonus)** Le système doit intégrer un module de 3D Gaussian Splatting alimenté par les poses estimées par le SLAM.

4.2 Besoins non fonctionnels

- BNF-1 Performance** : le pipeline SLAM doit fonctionner à une fréquence minimale de 10 Hz sur la plateforme embarquée.
- BNF-2 Portabilité** : le code doit être compatible Ubuntu 22.04 / ROS 2 Humble.

BNF-3 Reproductibilité : l’environnement de développement doit être containerisé (Docker) et versionné (Git).

BNF-4 Maintenabilité : le code doit respecter les conventions PEP 8 (Python) et les guidelines ROS 2 (C++ / Python).

BNF-5 Sécurité : le robot doit s’arrêter automatiquement en cas d’obstacle détecté à moins de 30 cm (sécurité intégrée TurtleBot 4).

4.3 Cas de tests

Table 4.1: Plan de tests fonctionnels

ID	BF associé	Scénario	Critère de succès
T-01	BF-1	Téléopérer le robot sur 5 m en ligne droite	Déviation < 5 cm
T-02	BF-3	Calibration caméra sur davier 8×6	Erreur de reprojection < 0.5 px
T-03	BF-4	SLAM RGB sur séquence corridor 30 m	ATE < 0.1 m
T-04	BF-5	SLAM LiDAR sur même séquence	ATE < 0.05 m
T-05	BF-6	Comparaison RGB vs LiDAR	Rapport d’évaluation généré

Chapitre 5

Choix logiciels

5.1 Système d'exploitation et middleware

Ubuntu 22.04 LTS Distribution de référence pour ROS 2 Humble, bénéficiant d'un support long terme et d'une large communauté robotique.

ROS 2 Humble Hawksbill Version LTS de ROS 2 offrant une architecture DDS fiable, des outils de débogage matures (RViz2, rqt, ros2bag) et une compatibilité native avec TurtleBot 4.

5.2 Algorithmes de SLAM

RTAB-Map Choix principal pour le SLAM multi-capteurs. Supporte nativement RGB-D, LiDAR 2D/3D et la fusion multi-capteurs. Expose des interfaces ROS 2 complètes et permet un réglage fin via ses nombreux paramètres.

KISS-ICP Utilisé comme baseline LiDAR pour sa robustesse et sa facilité de déploiement.

ORB-SLAM3 Utilisé pour la branche SLAM visuel pur (RGB) afin de disposer d'une référence académique reconnue.

5.3 Calibration

camera_calibration (ROS 2) Calibration intrinsèque caméra via damier (modèle pinhole + Brown-Conrady).

Kalibr Calibration extrinsèque caméra/LiDAR et calibration visuo-inertielle (caméra + IMU).

5.4 Évaluation

evo Bibliothèque Python de référence pour l'évaluation des trajectoires SLAM (ATE, RPE, visualisation).

TUM RGB-D Benchmark tools Pour la génération de vérités terrain (*ground truth*) comparables.

5.5 Développement et versioning

Python 3.10 / C++17 Langages principaux imposés par ROS 2 Humble.

Git + GitHub Versioning et collaboration.

Docker Containerisation de l'environnement pour la reproductibilité.

VS Code + ROS extension Environnement de développement intégré.

Chapitre 6

Architecture

6.1 Architecture matérielle

La plateforme TurtleBot 4 est composée des éléments matériels suivants :

- **Base iRobot Create 3** : plateforme de déplacement différentiel avec encodeurs et [IMU](#) intégrés.
- **Raspberry Pi 4B (4 Go)** : ordinateur embarqué principal.
- **Caméra OAK-D Pro** : caméra RGB-D stéréo avec accélérateur neural intégré (Intel MyriadX).
- **LiDAR RPLiDAR A1M8** : scanner laser 2D 360° à 12 Hz.
- **(Bonus) Caméra LWIR FLIR Lepton 3.5** : micro-caméra thermique 160×120 pixels, 8.7 Hz.

6.2 Architecture logicielle ROS 2

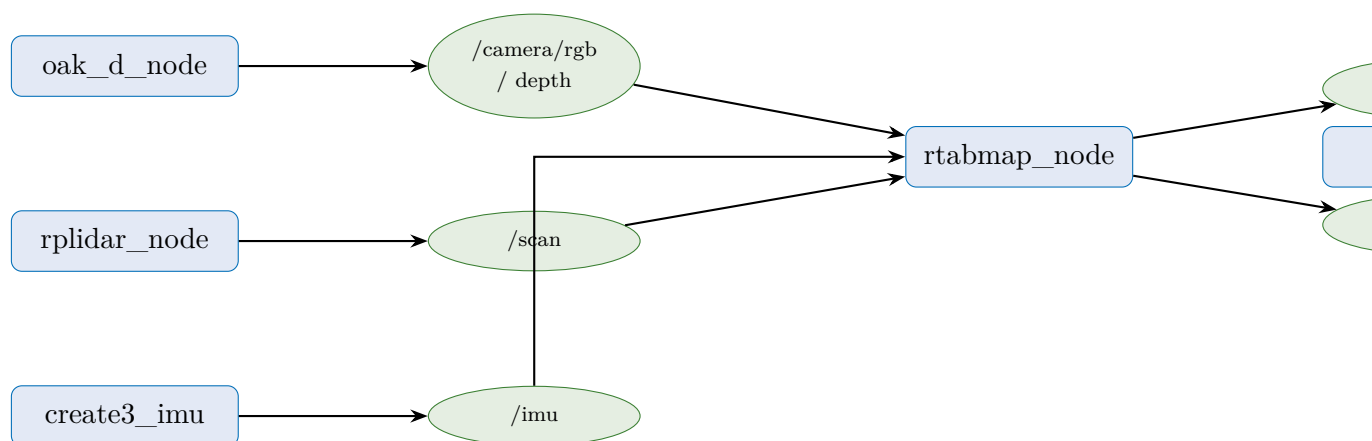


Figure 6.1: Graphe des nœuds et topics ROS 2 du pipeline MC-SLAM

6.3 Diagramme de cas d'utilisation (UML)

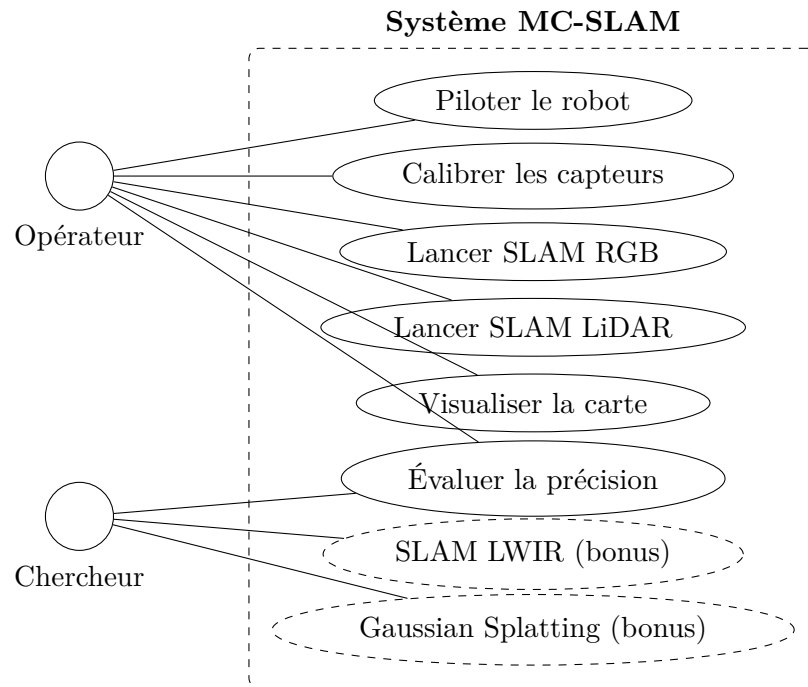


Figure 6.2: Diagramme de cas d'utilisation UML

Chapitre 7

Réalisation

7.1 Mise en place de l'environnement

L'environnement de développement a été configuré comme suit:

```
1
2 sudo apt update && sudo apt install ros-humble-desktop -y
3 sudo apt install ros-humble-turtlebot4-desktop -y
4
5 git clone https://github.com/xdZireael/PFE
6
7 cd mc_slam_ws && colcon build --symlink-install
8
9 source install/setup.bash
```

Listing 7.1: Installation de l'environnement ROS 2 et TurtleBot 4

7.2 Screenshots



(a) Carte SLAM RGB dans RViz2



(b) Carte SLAM LiDAR dans RViz2

Figure 7.1: Visualisation des cartes générées par les deux pipelines SLAM

7.3 Algorithmes et méthodes

7.3.1 Pipeline SLAM RTAB-Map

ICP from scratch:

Algorithm 1 ICP (Iterative Closest Point) pour l'odométrie LiDAR

Require: $P = \{p_i\}$, $Q = \{q_j\}$ (nuages de points source et cible)

Ensure: Transformation rigide $T \in SE(3)$ alignant P sur Q

- 1: Initialiser $T \leftarrow I$ (identité)
- 2: **repeat**
- 3: Trouver les correspondances $C = \{(p_i, q_j)\}$
- 4: Estimer la transformation T minimisant l'erreur de correspondance:

$$T = \arg \min_T \sum_{(p_i, q_j) \in C} \|Tp_i - q_j\|^2$$

- 5: Appliquer T à P : $P \leftarrow TP$
 - 6: **until** convergence (changement de T inférieur à un seuil) ou nombre maximal d'itérations atteint
-

7.3.2 Calibration extrinsèque caméra/LiDAR

La calibration extrinsèque consiste à estimer la transformation rigide ${}^L\mathbf{T}_C \in SE(3)$ entre le repère LiDAR $\mathcal{F}_L$ et le repère caméra $\mathcal{F}_C$:

$${}^L\mathbf{T}_C = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix} \in SE(3) \quad (7.1)$$

Chapitre 8

Résultats et tests

8.1 Résultats expérimentaux

Table 8.1: Comparaison des performances SLAM sur la séquence de test

Méthode	ATE RMSE (m)	RPE RMSE (m)	Fréquence (Hz)	Ferm
ORB-SLAM3 (RGB)	—	—	—	
KISS-ICP (LiDAR)	—	—	—	
RTAB-Map (RGB+LiDAR)	—	—	—	
RTAB-Map (LWIR)	—	—	—	

Chapitre 9

Conclusion

9.1 Bilan

Ce projet a permis PLACEHOLDER. La plateforme TurtleBot 4 s'est avérée PLACEHOLDER à prendre en main, notamment en ce qui concerne la calibration multi-capteurs. Le pipeline MC-SLAM développé sous ROS 2 montre PLACEHOLDER sur les séquences testées.

9.2 Perspectives

- **Temps réel embarqué** : optimiser les algorithmes pour les contraintes du Raspberry Pi 4 (TensorRT, ONNX Runtime).
- **SLAM sémantique** : intégrer la reconnaissance d'objets pour une cartographie sémantique.
- **Navigation autonome** : coupler le SLAM à un planificateur de chemin (Nav2) pour l'exploration autonome.
- **3D Gaussian Splatting** : finaliser l'intégration et évaluer la qualité de rendu sur les cartes construites.

Références bibliographiques

- [1] C. CAMPOS, R. ELVIRA, J. J. G. RODRÍGUEZ, J. M. M. MONTIEL et J. D. TARDÓS, “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” *IEEE Transactions on Robotics*, t. 37, n° 6, p. 1874-1890, 2021.
- [2] Y. WU, L. WANG, L. ZHANG et al., “Improving Autonomous Detection in Dynamic Environments with Robust Monocular Thermal SLAM System,” *ISPRS Journal of Photogrammetry and Remote Sensing*, t. 203, p. 265-284, 2023, ISSN : 0924-2716. DOI : [10.1016/j.isprsjprs.2023.08.002](https://doi.org/10.1016/j.isprsjprs.2023.08.002).